# 程序设计综合实验

## 第 2 周：斐波那契、特征值与排序

黄韬
taohonker@gmail.com
昆明理工大学·系统科学系

2026-09-23

# 目录

1 课前 . . . 3
1.1 本周提要 . . . 3
1.2 关于讲义与课程内容 . . . 4
1.3 关于作业 . . . 5
1.4 奖励任务 . . . 6
1.5 课程安排 . . . 7
1.6 考核与评分 . . . 8

**斐波那契、特征值与排序** **9**

1 递归算法 . . . 10
1.1 斐波那契数列的递归定义 . . . 10
1.2 递归法一：直接照着递推式写 . . . 11
1.3 递归法二：递归对 . . . 13
2 Fibonacci 数列 . . . 14
2.1 数列的封闭式与增长速率 . . . 15
2.2 精度边界：Double 与 Int64 . . . 16
3 特征值 . . . 17
3.1 矩阵法：把递推写成矩阵幂 . . . 18
3.2 特征值分解 . . . 19
3.3 特征分析法 . . . 21
4 排序算法 . . . 23
4.1 排序：一条新闻 . . . 23
4.2 冒泡排序 . . . 24
4.3 插入排序 . . . 26
4.4 快速排序 . . . 27
4.5 三种排序的性能对比 . . . 31
5 作业 . . . 33
5.1 作业要求 . . . 34
5.2 作业一：四种斐波那契算法 . . . 35
5.3 作业二：C 函数库与 Python 调用 . . . 36
5.4 作业三：用指针数组搭一个上三角矩阵 . . . 37
6 参考资料 . . . 38

# 课前

## 本周提要

- 从 Fibonacci 数列的计算看递归思想与算法复杂性
- 在数组上实现几种排序算法
- 从快速排序算法看分治思想

## 关于讲义与课程内容

- 讲义是自编的，不正式，勿外传
- 讲义里的 Julia 代码都在 julia 1.12 上跑过，**计时数据是本机实测值**，换一台机器数字会变，量级与增长趋势不变

Q 1 AI 1



Q 1 理论课、实验课、上机考试、期末考试分别占总评的多少？上机考试的作答条件（开卷还是闭卷、能否联网、几次机会）是什么？

AI 1 请让 AI 帮助整理：C、Python、Julia 三种语言在数值计算上的典型分工（编译方式、类型系统、数组与内存布局各有什么差别），并说明为什么同一段递推代码在三种语言里的耗时能差出几个数量级。

## 关于作业

- 理论课每周课后布置一次 Homework，**统一次周周二晚上提交**
- 实验课共有 6 个 Lab、十余个检查点（checkpoint），平均 1-2 周提交一次
- 理论课 Homework 与实验课 Lab 重复的部分可以一份交两遍
- 提交的至少是程序源文件，需要说明的部分写在注释里；特别复杂的解释与运行结果的可视化才用 Markdown
- 【方括号包围的作业项目】是【选作】
- 奖励任务在给定期限前提交，可以收获期末额外分数奖励

Q 2 AI 2



Q 2 一次编程作业如果只交一个 Markdown 说明文件、里面贴了几段代码截图，为什么会被扣分？按讲义的要求，一次作业至少要包含什么？

AI 2 请让 AI 帮助查找：把算法实现编译成函数库供另一种语言调用，工程上一般有哪几种方式（如 C 动态库、FFI、Cython、juliacall 等），各自要付出什么代价，以及跨语言调用时数据传递的开销通常体现在哪些环节。

## 奖励任务

- 理解并注释随机数发生器的代码与算法，3 分
  - 只限于 Xorshift, Xoshiro, MT19937_64 三种算法
  - 参考：Numerical Recipes, 3rd Ed.；TAOCP, vol. 2
- kNN 近邻算法，2 分
- Voronoi 剖分算法，1 分
- Dijkstra 最短路径算法，1 分
- 以上算法的并行版本另有加分，小于 4 分
- 花 1 课时为大家讲解 VTune 的用法，5 分
- 花 20 分钟为大家讲解 python profiler，3 分
- 提交期限以课程网站公告为准

AI 3



AI 3 请让 AI 帮助调研：MT19937_64 与 xoshiro 两类伪随机数发生器的状态有多少位、生成一个 64 位随机数要做哪些位运算、周期长度各是多少，并说明为什么数组性能测试必须用质量合格的随机数发生器。

## 课程安排

- 理论课：周四，3-5 节，多媒体 322
- 实验课：周四，1-2 节，理 108、110
- 助教：肖竹飞（理 418-2）
- 课程网站：https://git2.taohonker.space
- 作业与考核（两级权重见“考核与评分”一节）
  - 平时成绩 30%：常规作业 60% + **个人 DSA 库 20%** + 甘特图大作业 20%
  - 机试成绩 30%：期中 40% + 期末 60%
  - 期末笔试 40%：闭卷
  - 总评 ≥ 60 分为合格

**笔记 1：作业要求**

- 每次课上布置，无额外说明时**次周周二晚上**提交，具体时间以课程网站公告为准。
- 作业形式多样，包括编程题、理论题、AI 探究题等。**手写作业需拍照上传**，不要丢弃，期末统一收取。
- 本课程的教学语言是 **C 与 Julia**。题目指定语言的，务必用指定语言完成；题目没有语言要求的，任选 C 或 Julia；使用其它语言导致评分出错的责任自负。
- 编程作业请务必保证代码**可运行**，且输出结果正确，代码风格良好，**有注释**。可使用 Markdown 文件或 Jupyter Notebook 等格式提交展示运行结果，但请**务必包含源代码文件**（可以是 `.c`, `.h`, `.jl`, `.ipynb` 格式）。非必要**请勿附加数据文件与图片**，控制作业体积。
- 作业提交：使用课程 Git 平台（https://git2.taohonker.space）提交。每次作业独立 fork 教师仓库（不改名），本地开发保留完整 Git 记录，导出为独立作业仓库，推送前必须先 pull 同步自己的 fork，再 push。详细规则见课程网站的作业提交说明。提交时有冲突、未按格式提交、未在截止时间前提交均评为 D。未提交不予评分。
- 不要完全依赖 AI 工具，**上机考试时会断网，没有重考机会！**



Q 3 AI 4

Q 3 本周作业的截止时间是什么时候（无额外说明时）？如果本周的说明里另写了时间，以哪一个为准？

## 考核与评分

### 理论课（考试课，3 学分）

| 环节 | 权重 | 内容 |
|---|---|---|
| 平时成绩 | 30% | 常规作业 60% + 个人 DSA 库 20% + 甘特图大作业 20% |
| 机试成绩 | 30% | 期中 40% + 期末 60% |
| 期末笔试 | 40% | 闭卷 |

- **个人 DSA 库**：把课程里讲过的数据结构、基本算法逐个实现、组织成自己的库（接口、测试、复杂度记录都留痕），学期末整体评分。
- 课堂表现是**扣分项、没有基础分**：无故缺课或被公开警告，一次扣 5 分，扣完为止。

详解 1 Q 4 AI 5

### 实验课（考查课，2 学分）

- 没有环节权重：每个检查点独立按百分制打分，**最终成绩 = 各检查点得分的算术平均**。
- 每个实验交一份报告：目的、过程、结果与分析（复杂度、数据），以及 **AI 使用与验证过程的记录**。分析深度是拿到 90+ 的必要条件。

### 考试与作业的语言

- 教学语言是 **C 与 Julia**：题目指定语言的按指定，**没有语言要求的任选**。
- 机试禁用直接解题的标准库函数，算法必须手写；只实现题目要求的函数，不需要写 `main`。



详解 1

### 四档评分标准、命题蓝图与考核分流

作业（占平时成绩的 60%）：

| 得分 | 描述 |
|---|---|
| < 60 | 抄袭、未按时完成、代码无法运行或思路混乱 |
| 60-75 | 独立思考、按时完成，代码基本可运行，思路基本清晰 |
| 75-90 | 独立思考、按时完成，代码规范可运行，思路清晰，复杂度分析基本正确 |
| 90-100 | 独立思考、按时完成，代码结构优秀且有优化，分析严谨，体现工程规范 |

机试：

| 得分 | 描述 |
|---|---|
| < 60 | 无法完成基本功能，程序无法运行或逻辑错误严重 |
| 60-75 | 完成基本功能，存在部分错误或实现不规范，运行结果基本正确 |
| 75-90 | 功能完整，结构清晰，能处理边界情形，运行正确 |
| 90-100 | 功能完整，实现高效，结构优秀，测试充分，体现独立设计能力 |

AI 4 请让 AI 帮助说明：在 Git 平台上一个“fork 教师仓库、本地开发、导出独立仓库、先 pull 再 push”的作业提交流程，每一步各自解决了什么问题，以及不 pull 直接 push 会发生什么。

Q 4 理论课总评里“个人 DSA 库”占多少？请说出它折算进总评的两级权重。课堂表现扣分与它的关系是什么？

AI 5 请让 AI 帮助说明：高校课程常用的“达成度评价”（OBE）怎么从一个班的成绩单算出各课程目标的达成度，并解释为什么达成度可能大于 100%。

期末笔试（闭卷）：

| 得分 | 描述 |
| --- | --- |
| < 60 | 答题不完整，概念混淆，算法设计错误较多 |
| 60-75 | 基本概念清楚，部分题目有误，算法设计基本正确但表达不够清晰 |
| 75-90 | 概念与原理清楚，答题完整，算法设计正确，能够进行复杂度分析 |
| 90-100 | 概念体系完整，算法设计合理高效，分析严谨，具有较强的综合能力 |

实验检查点：功能完整、运行正确、能处理边界情形落在 75-90；结构优秀、有优化、测试充分、运行稳定高效落在 90-100。

**笔试命题蓝图**：识记/理解 30%（概念辨析、复杂度判断、算法步骤填空）、应用/分析 40%（算法执行 trace、存储表示分析）、综合/设计 30%（给定场景选结构与算法并论证）。

**考核分流**：核心考点约 70%（链表、排序、树、图、DFS/BFS、Dijkstra、动态规划基础）；认知拓展约 30%（桶排序、散列表、元启发式算法概念、KD 树/四叉树）。

# 斐波那契、特征值与排序

# 递归算法

## 斐波那契数列的递归定义

$$F(n) = F(n-1) + F(n-2), \qquad F(1) = 1, \quad F(2) = 1$$

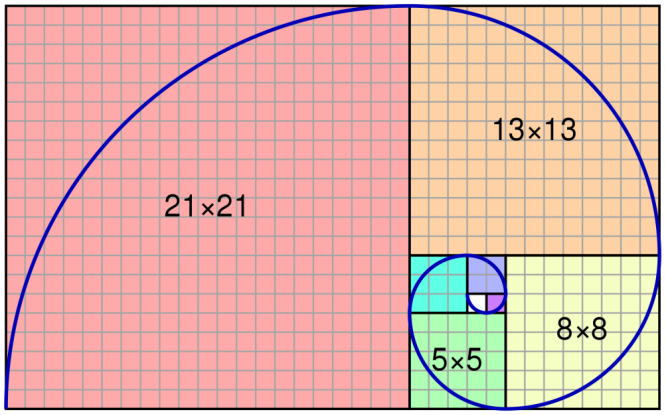



图 1：斐波那契数列拼成的正方形与黄金螺旋

- 这个递推式是整个学期最简单、也最有用的一个例子：**递归**、**算法复杂性**、**矩阵与特征值**三件事都能在它身上讲完。
- 先用你熟悉的语言把它实现出来，算出 $F(30)$：答案 $F(30) = 832\,040$。

Q 5 AI 6

Q 5 不用程序，按递推式手算出 F(10)，并说明你的算法每一步做了多少次加法。

## 递归法一：直接照着递推式写

通常用**递归法**计算 $F(n)$ 来衡量一种语言或一种计算芯片的数值性能：代码只有一行，谁来写都一样，因此比出来的就是平台本身。

（本节的计时与画图脚本见 `examples/fib.jl`，四种方法的代码与实测数字都能复现；纳秒级的数字在不同机器与不同次运行之间会有几成浮动。）

```
fib(n) = n ≤ 2 ? 1 : fib(n - 1) + fib(n - 2)

fib(30)      # 832040
@btime fib(30)   # 2.223 ms
```

- 实测 $F(30)$ 要 2.223 ms；换到 $F(45)$ 要 **3.063 s**。
- $n$ 每增加 1，耗时乘约 $\varphi = 1.618$，所以时间是随 $n$ 指数增长的。

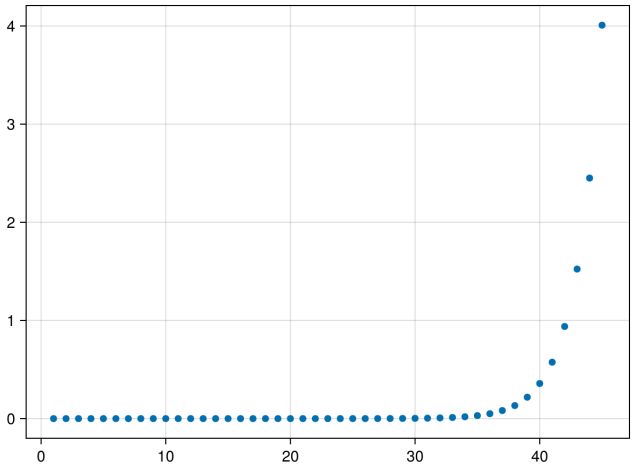



图 2: 递归法一的实测耗时随 n 的变化



AI 6 请让 AI 帮助查找：斐波那契数列在自然界与几何中出现的实例（如叶序、螺旋、蜂群家系），并说明这些实例里出现的究竟是数列本身，还是它的相邻项之比趋近的那个常数。

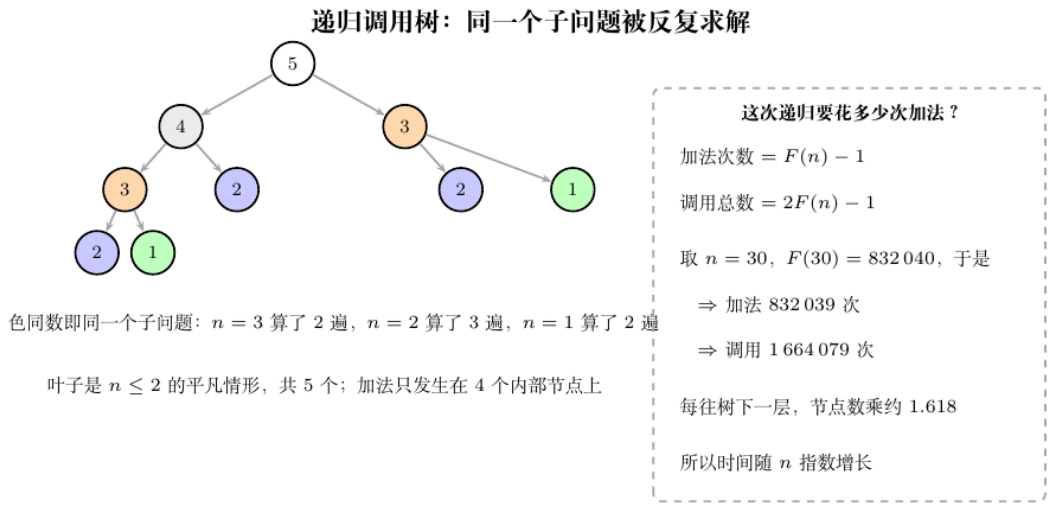



图 3: 斐波那契递归调用的调用树与加法次数

详解 2 Q 6 AI 7



详解 2

### 加法次数到底是多少

数一数调用树：$n \le 2$ 的叶子是平凡情形，不需要加法；加法只发生在内部节点上。

设 $A(n)$ 为算 $F(n)$ 需要的加法次数，则

$$A(1)=A(2)=0, \qquad A(n)=A(n-1)+A(n-2)+1 \quad (n \ge 3). \tag{0-2-1}$$

令 $B(n)=A(n)+1$，上式变成 $B(n)=B(n-1)+B(n-2)$，而 $B(1)=B(2)=1$，正是斐波那契数列的定义，于是

$$A(n)=F(n)-1, \tag{0-2-2}$$

调用总次数则是 $2F(n)-1$。

取 $n=30$：加法 $A(30)=832\,039$ 次，函数调用 $1\,664\,079$ 次，而结果只占一个整数。这就是递归法一慢在哪里：**同一个子问题被反复求解**。$F(30)$ 只被调用 1 次，$F(28)$ 也只被调用 2 次；但越靠下的项被重复得越厉害：$F(3)$ 被调用 $317\,811=F(28)$ 次，$F(2)$ 被调用 $514\,229=F(29)$ 次。

Q 6 按 A(n) = F(n) - 1，算 F(31) 比算 F(30) 多出多少次加法？为什么这个答案与“耗时乘 1.618”是同一件事的两种说法？

AI 7 请让 AI 帮助说明：如果把每个已算过的 F(k) 存下来复用（记忆化），函数的调用次数会降到多少，空间要多大，并解释这种做法与下一节的递归对方法在思路上有什么共同点。

## 递归法二：递归对

前一种写法的问题在于“一次只要一项”。把两项打包成一个状态：

$$G(n) = (F(n+1), F(n)), \qquad \text{于是} \quad F(n) = G(n)_2.$$

（$G(n)_2$ 表示二元组的第 2 个分量，代码里就是 `G(n)[2]`。）这样 $G(n)$ 只依赖 $G(n-1)$，递推关系里不再出现隔项：

$$G(n) = \big(\text{sum}\,(\,G(n-1)\,), G(n-1)_1\big), \qquad G(1) = (1, 1).$$

于是只要算 $n$ 次就能得到 $G(n)$，进而得到 $F(n)$。



```julia
function g(n)
  if n == 1
    return (1, 1)
  end

  tup = g(n - 1)
  return (tup[1] + tup[2], tup[1])
end

fib(45)      # 1134903170, @btime 3.063 s
g(44)        # (1134903170, 701408733)
@btime fib(45)   # 3.063 s
@btime g(44)     # 90.5 ns
```

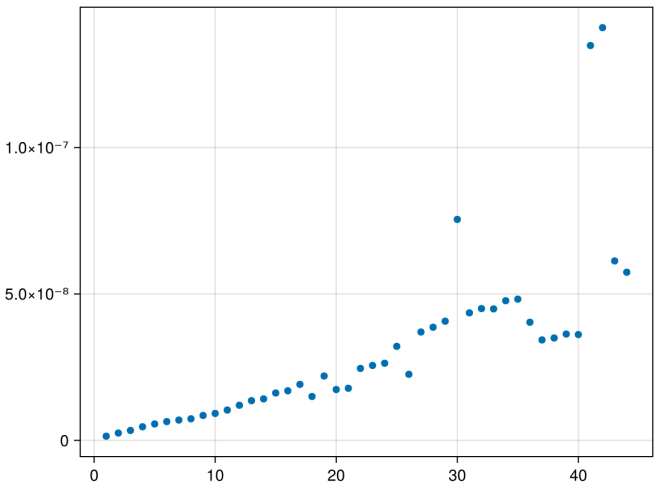



图 4: 递归法二的实测耗时随 n 的变化

- 耗时落在 ns 级别
- 随 $n$ **线性**增长：$n$ 加 1 只多一次函数调用

详解 3 Q 7 AI 8

详解 3

**为什么多带一项就能把指数降成线性**

递归的调用次数等于“被求解的子问题个数”。递归法一在算 $F(n)$ 时把 $F(n-1)$ 与 $F(n-2)$ 各算一遍，它们的子问题大量重叠，重叠部分每层翻一倍，于是个数随 $\varphi^n$ 增长。

递归法二每次只依赖一**个**前驱状态，从 $G(1)$ 走到 $G(n)$ 一路一条链，没有分叉，所以调用 $n$ 次、做 $n$ 次加法，时间和附加空间都是 $O(n)$。

同一个问题、同一个递推式，只因为“一次算一项”与“一次算一对”，复杂度从 $O(\varphi^n)$ 落到 $O(n)$。这是本周最值得记住的一件事：**先想清楚要算的状态是什么，再去写递归**。

Q 7 G(44) 给出的两个数是 1134903170 与 701408733，它们分别是 F 的哪两项？如果要算 F(46)，用递归法二只需要改哪一处？

AI 8 请让 AI 帮助解释：递归法二的递归调用是不是尾递归，编译器（或语言运行时）在什么条件下能把尾递归改写成一串循环，以及这种改写为什么能省掉调用栈。

## Fibonacci 数列

## 数列的封闭式与增长速率

上面两种写法都在做“一步一步算”。那么这个数列本身有没有封闭式，也就是一个直接由 $n$ 算出 $F(n)$ 的公式？有，而且它的两个底数就是黄金比：

$$\varphi = \frac{1+\sqrt{5}}{2} = 1.6180339887\ldots, \qquad \psi = \frac{1-\sqrt{5}}{2} = -0.6180339887\ldots = -\frac{1}{\varphi}$$

$$F(n) = \frac{\varphi^n - \psi^n}{\sqrt{5}}.$$

- 各项之比 $F(n+1)/F(n)$ 随 $n$ 增大趋向 $\varphi$，这就是“$n$ 每加 1，工作量乘约 1.618”的来源。
- 因为 $|\psi| < 1$，$\psi^n$ 很快就小到可以忽略：$F(n)$ 就是 $\varphi^n/\sqrt{5}$ 四舍五入到最近的整数。$F(30) = 832\,040$，而 $\varphi^{30}/\sqrt{5} = 832039.9947\ldots$。
- 封闭式把“求第 $n$ 项”变成了“算两个 $n$ 次幂”。这条路要先解决一个问题：$\varphi$ 与 $\psi$ 是从哪里来的？答案在下一节的**特征值**里。

Q 8 AI 9



**Q 8** 相邻项之比趋近 ， 那么 F(30)/F(29) 与 差多少？请用 的幂解释这个差为什么越来越小。

**AI 9** 请让 AI 帮助查找：封闭式 F(n) = (^n-^n)/5 的来历（Binet 公式）与它的等价形式，并说明这个公式在 n 很大时为什么不能直接用来算整数。

## 精度边界：Double 与 Int64

封闭式里全是浮点运算，这就给它划出了边界。本机实测（julia 1.12，与整数递推逐位对照，取整用 `round`；$G(n)$ 的两个分量分别是 $F(n+1)$ 与 $F(n)$）：

| $n$ | $F(n)$ 的偏差 | $F(n+1)$ 的偏差 |
|---|---|---|
| 70 | 0 | +1 |
| 78 | +22 | +37 |
| 90 | +8584 | +14523 |
| 92 | +22595 | 直接抛 `InexactError`（超出 Int64） |

- 偏差每加一层乘约 $\varphi = 1.618$。原因是双精度浮点只有约 16 位有效数字，而封闭式要求把 $\varphi^n/\sqrt{5}$ 四舍五入到整数；$F(n)$ 涨到 $10^{15}$ 以上（$n \geq 70$），这个四舍五入就不再保证正确。
- 换成整数路线就干净得多：递推式、递归对、矩阵幂都用 Int64 整数运算，能逐位精确算到 $F(92) = 7\,540\,113\,804\,746\,346\,429$。
- 所以“用封闭式把算法变成 $O(1)$”这句话要加一个条件：**在双精度还能分辨出整数位之前**。

详解 4 Q 9 AI 10



详解 4

### 误差是怎么长出来的，以及三条路线的边界各在哪里

完整对照表（特征值法重建值取整后与精确值之差）：

| $n$ | $F(n)$ 的偏差 | $F(n+1)$ 的偏差 |
|---|---|---|
| 70 | 0 | +1 |
| 72 | +1 | +2 |
| 78 | +22 | +37 |
| 82 | +161 | +263 |
| 90 | +8584 | +14523 |
| 92 | +22595 | 超出 Int64，`round(Int64, ...)` 直接抛 `InexactError` |

为什么偏差随 $n$ 一起长：$a_2^{n-1}$ 这一项的量级是 $\varphi^{n-1}$，每一次乘方都会引入约 $10^{-16}$ 的相对误差，于是绝对误差约等于 $F(n) \times 10^{-16}$，$n$ 每加 1 就跟着乘约 $\varphi$。实测从 $n = 78$ 的 22 涨到 $n = 90$ 的 8584，12 步涨了 390 倍，而 $\varphi^{12} = 322$，同一个量级。

还有一件事值得注意：$n$ 大时 Float64 的相邻可表示数之间的间隔（ulp）本身已经很大：$n = 90$ 那一行的量级是 $4.66 \times 10^{18}$，ulp $= 1024$。所以这几个整数与运算顺序有关，换一种等价写法就可能差 1 甚至差一个 ulp；看这张表要看**量级与趋势**，不必逐位较劲。

边界在哪里（Int64 能表示到 9 223 372 036 854 775 807）：

- **递归法 / 递归对**：只用整数加法，经实测逐位精确到 $n = 92$；$n = 93$ 起 $F(n)$ 超出 Int64，结果变成负数，这是**静默溢出**，编译器不会报错。

Q 9 如果只要求 F(n) 的前 10 位有效数字，封闭式能算到多大的 n？这和“要求精确到个位”的边界差在哪里？

AI 10 请让 AI 帮助整理：IEEE-754 双精度格式的位分配（符号位、指数位、尾数位）与由此决定的十进制有效位数，并解释 Int64 与 Float64 在能精确表示的整数范围上为什么差了三个数量级。

- **矩阵法**：$A^{n-1}$ 用 Int64 幂，实测同样精确到 $n = 92$；$n = 93$ 时算出的首元素是 $-6\,246\,583\,658\,587\,674\,878$，而正确值是 $F(93) = 12\,200\,160\,415\,121\,876\,738$：平方求幂的中间乘积先溢出，错误一路传到结果，同样不会报错。
- **特征值法**：$n \geq 70$ 起结果可能差 1；$n = 92$ 时 $F(93)$ 那一路大于 Int64 的上界，`round(Int64, ...)` 抛 `InexactError`，反倒是唯一会“喊出来”的一条路。

用更大的浮点类型（`BigFloat`）或高精度整数库（`BigInt`）可以把边界推后，代价是每次运算都慢几个数量级：这也解释了为什么工程上要同时关心“算法复杂度”与“数据类型的边界”。

## 特征值

## 矩阵法：把递推写成矩阵幂

把 $G(n) = (F(n+1), F(n))$ 写成矩阵形式，二阶递推就变成了一阶：

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix}, \qquad G_n = A\,G_{n-1} = A^{n-1} G_1 .$$

```julia
using LinearAlgebra
A = [1 1; 1 0]
G1 = [1, 1]
G(n) = A^(n - 1) * G1

G(44)        # 1134903170, 701408733
@btime G(44) # 260.3 ns
```

```julia
using StaticArrays
sA = @SMatrix [1 1; 1 0]
sG1 = @SVector [1, 1]
sG(n) = sA ^ (n - 1) * sG1

sG(44)        # 1134903170, 701408733
@btime sG(44) # 46.3 ns
```

详解 5 详解 6 AI 11



详解 5

### 矩阵幂要做多少次计算

不要用连乘写 $A^{n-1}$：那是 $n-2$ 次 $2\times 2$ 矩阵乘。Julia 的 `^` 用的是**平方求幂**，次数降到约 $2\log_2 n$ 次矩阵乘（$n=44$ 时约 10 次）。

两者都是“常数级”的乘法次数与 $n$ 的对数有关，剩下的差别全在常数上：

- 普通 `Matrix` 每次乘法都要在堆上分配结果，实测 $G(44)$ 要 260.3 ns，其中大部分是分配与垃圾回收的开销。
- 定长矩阵 `SMatrix` 的尺寸写在类型里，乘法在编译期展开成几条指令，实测 $sG(44)$ 只要 46.3 ns。

所以矩阵法把“指数级”压到了“几十纳秒”；但它还是**矩阵**幂，每次乘法要算 4 个元素。下一节把矩阵换成一个数，能再快一档。

详解 6

### 什么叫定长（静态）数组，它为什么快

同一个词在三种语言里指的不是一回事：

| | 在 C 里 | 在 Python 里 | 在 Julia 里 |
|---|---|---|---|
| 名字 | 数组 `T a[N]` | `list` | `Vector{T}` |
| 长度 | 编译期固定的 N | 运行时可变（动态数组） | 运行时可变（动态数组） |

AI 11 请让 AI 帮助解释：定长数组（Julia 的 SMatrix、C++ 的 std::array 结合模板）为什么能把循环展开、把尺寸放进类型里，以及这种做法的典型代价（编译时间、代码体积）是什么。

| | 在 C 里 | 在 Python 里 | 在 Julia 里 |
|---|---|---|---|
| 存什么 | 连续排布的 T 本身 | **连续排布的指针**，指向堆上的对象 | 连续排布的 T 本身，带长度/容量字段 |
| 类型 | 编译期完全确定 | 每个元素可以是不同类型（都是 `PyObject*`） | 元素类型写进类型参数，运行时不查 |
| 定长版本 | 数组、结构体成员数组 | `tuple`（不可变、定长） | `NTuple`、`SVector{N,T}`、`SMatrix{N,M,T}`（StaticArrays） |

所以 Julia 的 `SMatrix{2,2,Int64}` 与 C 的定长数组是一类东西：**长度写在类型里**，编译器在做完类型推导后就知道要处理几个元素。它更快的原因有四条：

1. **尺寸是类型的一部分**：`sA * sB` 展开成固定条数的乘加指令（$2\times2$ 乘 $2\times2$ 是 8 乘 4 加），没有循环、没有长度检查。
2. **不分配堆内存**：`SMatrix` 小于一定尺寸时直接放在栈上甚至寄存器里，没有 GC 压力；普通 `Matrix` 每次乘法都要 `malloc` 一块结果（实测 $G(44)$ 有 20 次分配，$sG(44)$ 只有 2 次）。
3. **类型确定 → 无动态派发**：Julia 按类型特化编译；元素类型不确定（如 `Vector{Any}`）或全局变量未标注时，每次运算都要在运行时查类型。
4. **数据紧凑 → 缓存友好**：连续排布意味着一次 cache line 能装下整个矩阵。

代价：定长数组的长度必须在编译期知道，尺寸一变就要重新编译一份代码（编译时间与代码体积都会涨），所以它只适合维度小而且固定的场合，比如 $2\times2$ 的斐波那契矩阵。

实测对照（同一台机器）：$G(44)$ 用普通 `Matrix` 是 260 ns，$sG(44)$ 用 `SMatrix` 是 46 ns，差 5 倍多。

## 特征值分解

矩阵 $A=\begin{pmatrix}1&1\\1&0\end{pmatrix}$ 的特征值满足 $\lambda^2-\lambda-1=0$，也就是 $\lambda=\frac{1\pm\sqrt{5}}{2}$：**它自己的特征值就是黄金比与其共轭**。这解释了封闭式里的 $\varphi$ 与 $\psi$ 是从哪来的。

两组**特征对**（特征值 + 对应的单位特征向量）实测为：

$$a_1=-0.6180339887498949=-\frac{1}{\varphi},\qquad v_1=\begin{pmatrix}-0.5257311121191336\\0.8506508083520399\end{pmatrix},$$

$$a_2=1.618033988749895=\varphi,\qquad v_2=\begin{pmatrix}0.85065080835204\\0.5257311121191336\end{pmatrix}.$$

- 它们满足定义式 $Av_i=a_iv_i$（实测两边之差约 $5.6\times10^{-17}$，即双精度的舍入水平）。
- 两个向量都是单位向量（$|v_i|=1$）且互相垂直（$v_1\cdot v_2\approx0$）；$A$ 是对称矩阵，不同特征值的特征向量必然正交。
- 特征向量整体乘 $-1$ 仍是特征向量，所以不同软件给出的符号可能不同，这一点在下面的详解里说明。

```
egA = eigen(sA)

v1 = egA.vectors[:, 1]
# -0.5257311121191336, 0.8506508083520399
a1 = egA.values[1]       # -0.6180339887498949 = -1/φ

v2 = egA.vectors[:, 2]
# 0.85065080835204, 0.5257311121191336
```

```
a2 = egA.values[2]       # 1.618033988749895 = φ

l1 = dot(sG1, v1)        # 0.3249196962329063
l2 = dot(sG1, v2)        # 1.3763819204711736
```

详解 7 Q 10 AI 12



详解 7

**两组特征对是怎么求出来的**

**特征值**：特征多项式是 $\det(A-\lambda I)=(1-\lambda)(-\lambda)-1=\lambda^2-\lambda-1$，令它等于零：

$$\lambda^2-\lambda-1=0\Rightarrow\lambda=\frac{1\pm\sqrt{5}}{2}. \tag{0-4-3}$$

**特征向量**：把 $\lambda_2=\varphi$ 代回 $(A-\lambda I)v=0$，例如

$$\begin{pmatrix}1-\varphi & 1\\ 1 & -\varphi\end{pmatrix}v=0\Rightarrow v=t\,(\varphi,1)^T, \tag{0-4-4}$$

取 $t$ 让 $v$ 的模长为 1，就得到上面 `eigen` 给出的那一列。$\lambda_1=-1/\varphi$ 的那一列同理。

**系数**：$\{v_1,v_2\}$ 是一对线性无关的特征向量（它们对应的特征值不同），所以平面上任何向量都能写成它们的线性组合，$G_1$ 当然也可以：

$$G_1=\lambda_1 v_1+\lambda_2 v_2\Rightarrow\lambda_1=v_1\cdot G_1,\quad \lambda_2=v_2\cdot G_1, \tag{0-4-5}$$

最后一步用到了 $v_i$ 是单位向量、且 $v_1\perp v_2$（$A$ 是对称矩阵，不同特征值的特征向量正交）。这正是代码里 `dot(sG1, v1)` 算的东西。

**符号约定**：特征向量整体乘 $-1$ 仍是特征向量，$\lambda_i$ 也随之变号，这就是不同版本的特征值法里常数符号不同的原因；代入 $G_n$ 后结果完全一样。本节用的是 `eigen` 给出的这一组。

Q 10 A 是对称矩阵, 两个特征向量正交。如果把 v_1 整体乘 -1，_1 会怎么变？G_n 的表达式会不会因此改变？

AI 12 请让 AI 帮助列举：特征值分解在数据科学里的三个典型用途（如主成分分析、谱聚类、PageRank），并说明每一处用的都是“矩阵幂作用在向量上”这一模式。

## 特征分析法

有了特征对，矩阵幂就可以拆开：因为 $Av_i = a_i v_i$，所以 $A^{n-1}v_i = a_i^{n-1}v_i$。把 $G_1 = \lambda_1 v_1 + \lambda_2 v_2$ 代入 $G_n = A^{n-1}G_1$：

$$G_n = \lambda_1 a_1^{n-1} v_1 + \lambda_2 a_2^{n-1} v_2.$$

```julia
function GmInt2(n; v1 = v1, v2 = v2, a1 = a1, a2 = a2,
l1 = l1, l2 = l2)
  b1 = l1 * a1 ^ (n-1)
  b2 = l2 * a2 ^ (n-1)
  fn  = round(Int64, b1 * v1[1] + b2 * v2[1])
  fn2 = round(Int64, b1 * v1[2] + b2 * v2[2])
  return fn, fn2
end

GmInt2(44)      # (1134903170, 701408733)
@btime GmInt2(44) # 33.3 ns
```

- 33.3 ns，比递归法一的 3.063 s 快约 $9 \times 10^7$ 倍
- 与 $n$ 的关系只剩幂运算内部的对数因子，$n = 1$ 到 $n = 45$ 的实测曲线基本是平的

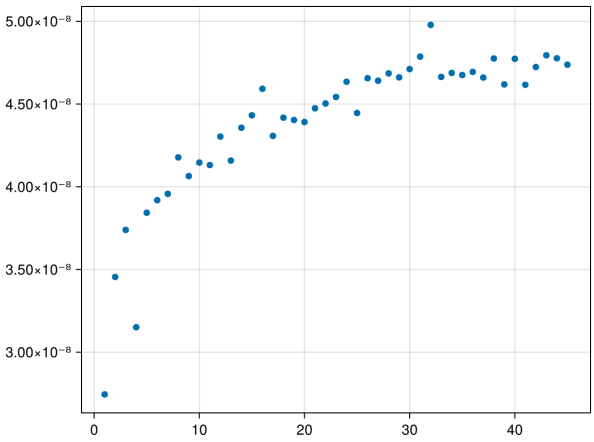



图 5: 特征值法的实测耗时随 n 的变化

- 表面上看问题已经解决了：求第 $n$ 项只要做两次幂运算，算法的时间复杂性被搬进了**数值计算**里。



- 取整必须用 `round`，不能用 `floor`：重建值带约 $10^{-16}$ 的相对误差，$n$ 小的时候它会落在整数下方（实测 $n = 1$ 时重建值是 0.9999999999999999，`floor` 给出 0，`round` 给出 1）。
- 代价是上一节量出来的精度边界（$n$ 到 70 之后结果不再逐位正确），以及“常数要在 $n$ 确定前先算好”这一步。

# 特征分析法

**笔记 2：扩展学习**

1. 学习 IEEE-754 标准关于数值算法的规定
2. Float64 的精度上限比 Int64 低
3. 学习 / 复习如何画图
    i. Julia: Makie.jl
    ii. Python: Matplotlib
    iii. MatLab
    iv. GNUPlot

**重要 1：程序慢的可能来源**

- 算法不行！
- 数据结构不行！
- 语言不行！
- 机器不行！
- 客户不行！
- 程序员不行！
- AI 不行！

AI 13



AI 13 请让 AI 帮助比较：Makie.jl、Matplotlib、GNUPlot 三者在“画一张带对数纵轴的散点图”这件事上各自的代码量与常见坑（中文字体、坐标轴刻度、导出 dpi），并给出三段可直接运行的示例。

# 排序算法

## 排序：一条新闻

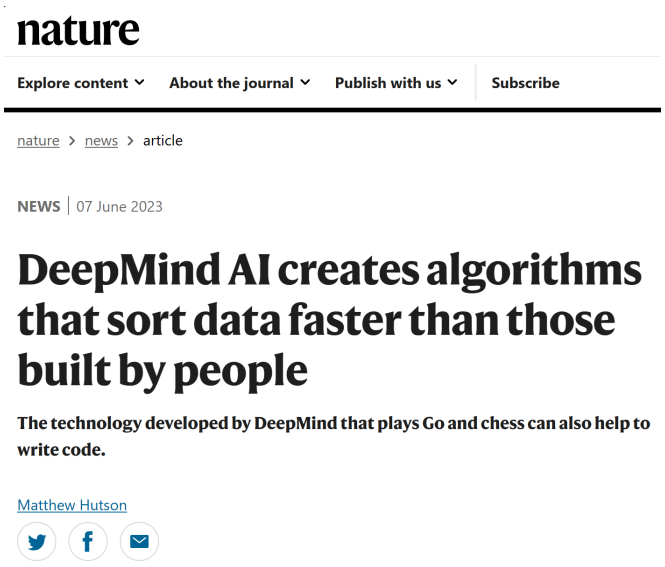
### 一条新闻

- AlphaDev（2023，DeepMind）：让 AI 从零合成排序代码，成果进了 LLVM 的标准库
  - 对短序列（3 到 5 个元素）排序，在汇编层面发现新的优化方式
  - 某些步骤不再用 `MAX(A,B,C)`，而是拆成 `MAX(A,B)` 的组合
  - 论文报告：短序列快约 70%，长序列（数十万级）约 1.7%
- 这条新闻说明了排序的两个侧面：**要理解算法本身**，也要理解它在具体机器上的常数。

### 参考

- TAOCP, vol. 3
- Algorithms, 4th Ed., Ch2
- Sorting Algorithms Animations

Q 11 AI 14



nature

Explore content ⌄ About the journal ⌄ Publish with us ⌄ Subscribe

nature › news › article

NEWS | 07 June 2023

**DeepMind AI creates algorithms that sort data faster than those built by people**

**The technology developed by DeepMind that plays Go and chess can also help to write code.**

Matthew Hutson

图 6: 《自然》对 AlphaDev 排序算法的报道截图

**Q 11** 为什么短序列上 70% 的提升，落到整个标准库上只写成 1.7%？请从“短序列占实际调用量的比例”这个角度解释。

## 冒泡排序




图 7：冒泡排序各轮扫描的过程示意

（完整脚本：`examples/bubble_sort.jl`。）



**AI 14** 请让 AI 帮助查证：AlphaDev 在 2023 年发表的排序算法论文（Nature）报告的加速比具体是在哪些长度的序列上测得的，代码合入了哪个开源工具链的哪个版本。

```julia
function bubble_sort!(v::AbstractVector)
  tmp = zero(eltype(v))
  len = length(v)
  for i in 1:len
    for j in 1:len-i
      if v[j] > v[j+1]
        tmp = v[j]
        v[j] = v[j+1]
        v[j+1] = tmp
      end
    end
  end
  return nothing
end
```

```julia
using BenchmarkTools

v = rand(10000)
bubble_sort!(v)
issorted(v)   # true

@btime bubble_sort!(rand(10000)) # 44.7 ms
@btime bubble_sort!(rand(100))   # 9.06 µs

@btime sort!(rand(10000)) # 128.5 µs
@btime sort!(rand(100))   # 1.61 µs
```

详解 8 Q 12 AI 15



详解 8

### 冒泡排序做了什么运算

两层循环各走一遍数组，比较次数固定是 $n(n-1)/2$，与输入是否有序无关；交换次数取决于逆序对的数量，随机输入下约为 $n^2/4$。

$n = 10000$ 时比较约 $5 \times 10^7$ 次，实测 44.7 ms，折合每次比较不到 1 ns，已经贴着内存带宽跑了。同时把数组缩小 100 倍，耗时降到 9.06 µs，说明这一层复杂度确实是平方级的。

Q 12 按现在的写法，即使数组已经有序也要把两层循环走完。请给出一个提前结束的判断条件，并说明它在“已排序输入”上能把比较次数降到多少。

AI 15 请让 AI 帮助说明：排序算法的“稳定性”指什么，冒泡排序与插入排序稳定还是不稳定（用代码里的交换条件说明），以及稳定性在实际数据处理里为什么重要。

## 插入排序

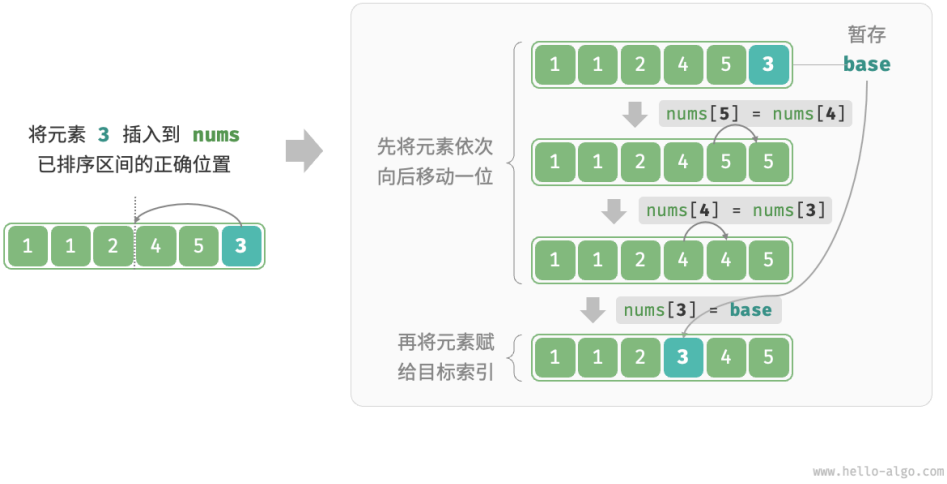



图 8: 插入排序一次插入的三步操作

(完整脚本：examples/insertion_sort.jl。)



```julia
function insertion_sort!(v::AbstractVector)
    len = length(v)
    for i in 2:len
        j = i
        x = v[i]
        while j > 1
            y = v[j-1]
            if x > y
                break
            end
            v[j] = y
            j -= 1
        end
        v[j] = x
    end
    return v
end
```

```julia
using BenchmarkTools

v = rand(1:100, 10000)
insertion_sort!(v)
issorted(v)   # true

@btime insertion_sort!(rand(10000)) # 9.5 ms
@btime insertion_sort!(rand(1000))  # 98.6 μs
@btime insertion_sort!(rand(100))   # 2.09 μs

@btime sort!(rand(10000)) # 128.4 μs
@btime sort!(rand(1000))  # 8.64 μs
@btime sort!(rand(100))   # 1.58 μs
```

详解 9 Q 13 AI 16

**详解 9**

### 插入排序的代价与输入分布的关系

内层循环把比 $x$ 大的元素整体后移一格，直到遇见一个不大于 $x$ 的元素。于是

- **已排序输入**：每个元素一进内层循环就 `break`，每个元素只比较 1 次，总共 $O(n)$ 次比较。实测 $n = 10000$ 只要 0.0117 ms。
- **逆序输入**：每个元素都要移到最前面，比较与移动都是 $O(n)$ 量级，总共 $O(n^2)$。实测 19.25 ms，比已排序输入慢约 1600 倍。
- **随机输入**：平均要把一个元素移过半个已排好的前缀，约 $n^2/4$。实测 9.64 ms。

同样一段代码，在已排序输入上能跑进 11.7 µs，比 Julia 内置的 `sort!` 在同样输入上的 8.88 µs 只差一点点；而在逆序输入上落后两个数量级。这就是下一节“三种排序的性能对比”要讲的第一个要点：**谈排序速度必须先说输入分布**。

**Q 13** 在你自己的机器上把 n 从 1000 加到一个更大的值，测出插入排序在已排序输入上的耗时。它与 n 大致成正比吗？与 sort! 相比谁更快？

**AI 16** 请让 AI 帮助分析：在随机输入下，冒泡排序与插入排序的交换次数为什么差不多，而比较次数不同（提示：注意内层循环的终止条件）。再让 AI 说明这个差别为什么会在实测里体现出来。

## 快速排序

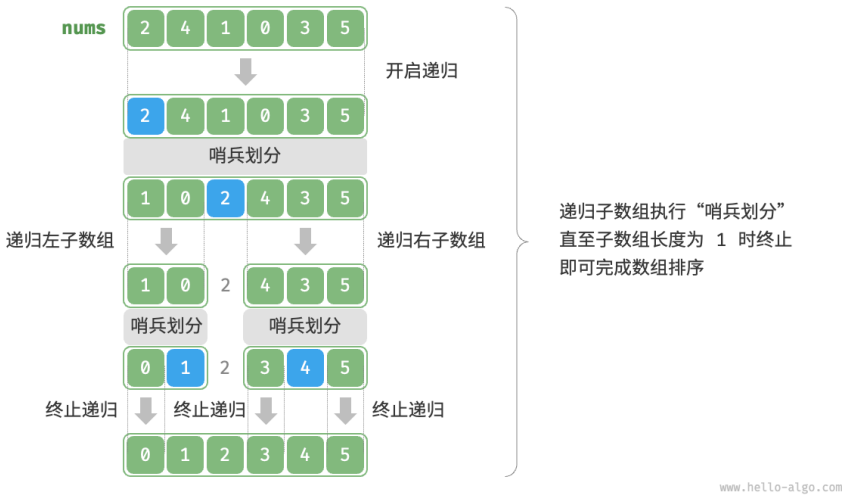



图 9: 快速排序以哨兵划分的分治流程

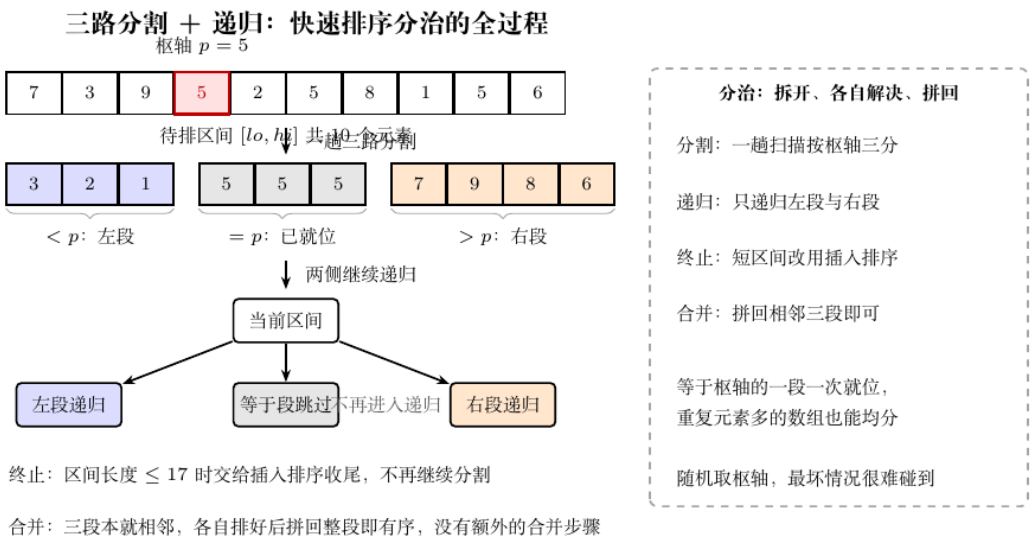



图 10: 快速排序三路分割与递归的分治过程

**接口**：对外只留一个函数，内部另分配一块同样大的辅助数组 $t$。

```
function quicksort!(v::AbstractVector)
  lo = 1
  hi = length(v)
```



```
  t = copy(v)
  quicksort!(v, lo, hi, t)
  return v
end
```

## 快速排序

**分治**：区间还长的时候分割成两半递归，短到一定程度就交给插入排序收尾。

```
function quicksort!(v::AbstractVector, lo, hi, t::AbstractVector)
  if hi - lo ≤ 16
    vv = @view v[lo:hi]
    insertion_sort!(vv)
    return nothing
  end

  lo1, hi1, lo2, hi2 = partition!(v, lo, hi, t)
  if lo1 < hi1
    quicksort!(v, lo1, hi1, t)
  end
  if lo2 < hi2
    quicksort!(v, lo2, hi2, t)
  end
  return nothing
end
```



## 快速排序

**3-路分割**：一趟扫描把区间按枢轴分成三段，等于枢轴的一段留在中间。分段时只用一块与区间等长的辅助数组 $t$：小于枢轴的从头部往后放，大于枢轴的从尾部往前放，等于枢轴的先不落位、最后整段填回去。

```
function partition!(v::AbstractVector, lo, hi, t::AbstractVector)
  ngreater = zero(lo)
  nequal = zero(lo)
  myone = one(lo)
  pivot_index = rand(lo:hi)   # 随机取枢轴，避免在已排序输入上退化
  pivot = v[pivot_index]

  @inbounds begin
    for i in lo:hi
      x = v[i]
      if x < pivot
        t[i-ngreater-nequal] = x
      elseif x > pivot
        t[hi-ngreater] = x
        ngreater += myone
      else
        nequal += myone
      end
    end
    t[hi-ngreater-nequal+myone:hi-ngreater] .= pivot
```

```
    v[lo:hi] .= t[lo:hi]
  end

  return lo, hi - ngreater - nequal, hi - ngreater + myone, hi
end
```



## 快速排序

```
using BenchmarkTools
# Float64
@btime quicksort!(rand(10000)) # 0.553 ms
@btime quicksort!(rand(1000))  # 42.4 μs
@btime quicksort!(rand(100))   # 2.77 μs
@btime sort!(rand(10000)) # 128.5 μs
@btime sort!(rand(1000))  # 8.55 μs
@btime sort!(rand(100))   # 1.57 μs
# Int64（取值范围只有 1:100，重复元素很多）
@btime quicksort!(rand(1:100, 10000)) # 251.3 μs
@btime quicksort!(rand(1:100, 1000))  # 30.3 μs
@btime quicksort!(rand(1:100, 100))   # 2.63 μs
@btime sort!(rand(1:100, 10000)) # 26.8 μs
@btime sort!(rand(1:100, 1000))  # 3.65 μs
@btime sort!(rand(1:100, 100))   # 596 ns
```

详解 10 Q 14 AI 17

**详解 10**

### 3-路分割的三个游标各是什么

一趟扫描结束时，区间 $[lo, hi]$ 被分成三段，由 4 个下标描述：

- `ngreater`：已经放进“大于枢轴”那一段的元素个数，它们从 $hi$ 往前放。
- `nequal`：等于枢轴的元素个数，它们**先不落位**，只是计数。
- 剩下的是小于枢轴的，从 $lo$ 往后放，落点是 `i - ngreater - nequal`：第 $i$ 个位置减去已经占掉的尾部与中部。
- 扫描结束后，把 $pivot$ 一次填满中段 `[hi-ngreater-nequal+1, hi-ngreater]`，再把整段拷回 $v$。

于是返回的 4 个数就是三段的下标：$(lo, hi - ngreater - nequal)$ 是小于段，$(hi - ngreater - nequal + 1, hi - ngreater)$ 是等于段（它已经就位、不必再排），$(hi - ngreater + 1, hi)$ 是大于段。

这样处理的好处是：数组里有大量重复值时，等于枢轴的整段一次排好，不会退化；配合随机取枢轴，最坏情况（$O(n^2)$）也很难碰到。

**Q 14** 代码里 hi - lo <= 16 这个阈值如果改成 2、或者改成 100，快速排序会有什么变化？请分别说明它花了更多时间在哪件事上。

**AI 17** 请让 AI 帮助比较：随机取枢轴、取首元素为枢轴、三数取中三种策略在“已排序输入”上的表现，并说明为什么随机取枢轴能把最坏情况的概率压得很低。

## 三种排序的性能对比

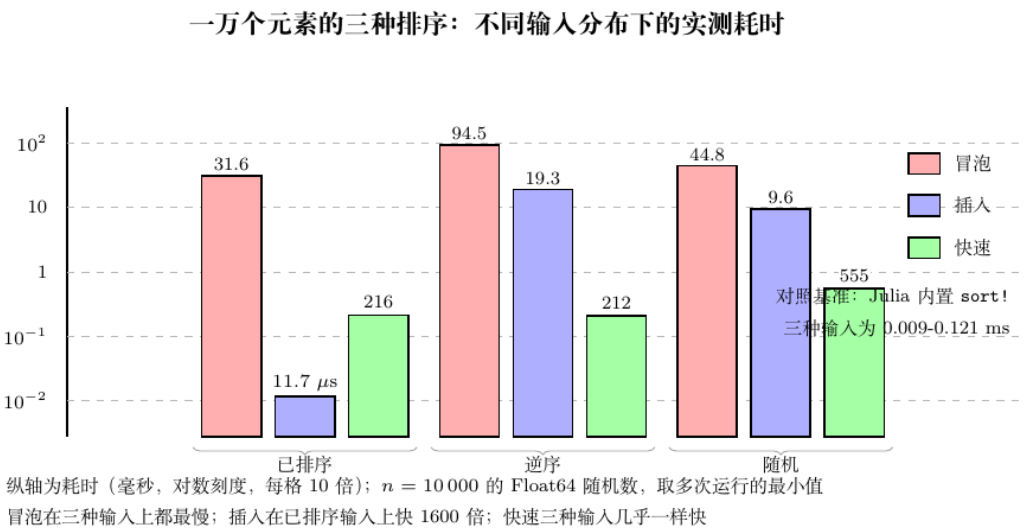



图 11：三种排序在三种输入分布下的实测耗时

同样的 $n = 10000$、同样的机器，只换输入分布（单位：ms；每个数字取多次运行的最小值，同一台机器上重复测量的抖动在几个百分点以内）：

| 算法 | 已排序 | 逆序 | 随机 |
|---|---|---|---|
| 冒泡排序 | 31.6 | 94.5 | 44.8 |
| 插入排序 | 0.0117 | 19.25 | 9.64 |
| 快速排序（3-路） | 0.216 | 0.212 | 0.555 |
| Julia 内置 `sort!` | 0.00888 | 0.01163 | 0.121 |

三句话总结这张表：

- **冒泡排序**在三种输入上都最慢。它的比较次数固定，且每次只交换相邻的两个元素，数据移动多。
- **插入排序**的结果完全看输入：已排序输入上每个元素只比较一次，比 `sort!` 只差一点；逆序输入上退化成平方级，比已排序输入慢 1600 倍。
- **快速排序**在三种输入上都很快，随机输入比已排序输入慢约 2.6 倍；两种输入的分割次数相近，差别在访存与分支预测上（已排序的数据是顺序访问，随机数据不是）。

详解 11 详解 12 Q 15 AI 18



Q 15 在你自己的机器上，把 n 从 10000 减小到 100，重测三种排序在随机输入上的耗时。三者之间的倍数关系变小了吗？为什么？

AI 18 请让 AI 帮助说明：基于比较的排序算法为什么有 O(n n) 的下界，快速排序在什么情况下会退化成 O(n^2)，以及计数排序、基数排序为什么能突破这个下界。

## 详解 11

### 从这三个算法的对比里能读出什么

1. **复杂度说的是增长趋势，不是绝对时间。** $O(n \log n)$ 与 $O(n^2)$ 的差别在 $n = 10000$ 时已经显现（0.555 ms 与 44.8 ms，差 80 倍），$n$ 再大十倍差距还会拉大。
2. **同一份代码在好输入上的常数可以差到很难相信。** 插入排序在已排序输入上是 11.7 µs，这个数字比快速排序在随机输入上的 555 µs 还快 47 倍；但没人会说插入排序比快速排序快。
3. **实现与算法同样重要。** “3-路分割 + 短区间用插入排序收尾 + 随机枢轴”这三处工程手段，把快速排序从教科书版本变成能贴着内置 `sort!` 跑的版本；内置 `sort!` 在随机输入上仍快 4-5 倍、在已排序与逆序输入上快 20 倍上下，差距在它更少的内存分配与更细的常数优化上。

## 详解 12

### 为什么 Julia 内置的 `sort!` 更快

它不是“一个算法”，而是一条**按输入特征选路的流水线**（`Base.Sort` 源码里的 `_DEFAULT_ALGORITHMS_FOR_VECTORS`，julia 1.12）：

```
先查是否已经有序 → 小数组（≤40）走专门路径 → 求最值 →
  元素能映射成 UInt 且值域小 → 计数排序（线性!）
                        值域稍大 → 基数排序
                否则（含一般数值、字符串等）→ 带临时缓冲的快速排序
```

这条流水线里有四件事是我们的 `quicksort!` 没有做的：

1. **有序检查**：先扫一遍判断是否已排序，是就直接返回。实测 $n = 10000$ 的**已排序** Float64 只要 8.88 μs（一次线性扫描），Int64 只要 2.53 μs；我们的快速排序在同样输入上要 216 μs（虽然也没退化，但白排了一遍）。
2. **按值域换算法**：整数元素的值域足够小时改用计数排序/基数排序，复杂度直接从 $O(n \log n)$ 变成 $O(n)$。实测（$n = 10000$，元素都是 Int64，只有取值范围不同）：

| 取值范围 | 实测 | 走的路线 |
|---|---|---|
| 1 : 100 | 6.3 μs | 计数排序，**每元素 0.0006 μs，与 n 成正比** |
| 1 : $10^6$ | 44 μs | 基数排序 |
| 1 : $10^9$ 与全范围 | 58 μs / 138 μs | 快速排序 |
| Float64 随机（对照） | 120 μs | 快速排序 |

注意 1:100 那一行：$n$ 从 $10^3$ 涨到 $10^6$，每元素耗时几乎不变，这正是线性算法的标志。我们手写的 3-路快速排序在同样输入上是 251 μs，比它慢 40 倍，差的不是常数，是算法的档次。

3. **小段收尾更细**：内置快排对 $\le 20$ 的小段（`SMALL_THRESHOLD`）直接用插入排序，而且用了带哨兵的变体减少边界判断；我们用的是 $\le 17$ 转插入排序，思路一致但阈值与写法更粗。
4. **临时缓冲复用**：内置的 `ScratchQuickSort` 在整棵树里复用一块缓冲；我们的 `partition!` 每一趟都要把整段 `t[lo:hi]` 拷回 `v`，等于每次分割多一次全区间内存写。

结论不是“我们写得没用”，而是：**教科书写法给出算法骨架，工程实现在骨架上叠加了对输入分布的判断、内存层次的利用和常数优化**。作业里比较“自己写的 vs 内置的”时，要把这一层说清楚，而不是只说“内置的快”。

## 作业

## 作业要求

- 本次作业使用 C 语言或 Python + C 的混合方式完成，也可以使用 Julia 语言完成
- 截止时间：无额外说明时为**次周周二晚上**，具体以课程网站公告为准
- 提交方式、命名与格式要求见“课前”一节的作业要求；**必须包含可运行的源代码文件**

Q 16 AI 19



Q 16 一份作业如果代码能跑、但没有任何注释、也没有说明它在什么输入下测过，按讲课时的评分口径应该落在哪一档？为什么？

AI 19 请让 AI 帮助说明：如何为“同一算法在两种语言里的实现”设计一份公平的计时对比（要固定哪些条件、重复多少次、报告最小值还是平均值），并指出常见的不公平做法。

# 作业一：四种斐波那契算法

使用 C 与 Python 分别实现 Fibonacci 序列的 4 种计算方法：

1. 递归法一（直接照着递推式写）
2. 递归法二（递归对）
3. 矩阵法（$G_n = A^{n-1}G_1$）
4. 特征值法（封闭式）

并作图：

- 比较两种语言中递归法一的速度
- 比较 Python 中递归法一与特征值法的速度

Q 17 AI 20



Q 17 特征值法只能算到 n 大约 72 之前还保证整数位全对。作业里如果要画“耗时随 n 变化”的曲线，四种方法各自的 n 应该取到多少才既看得出趋势、又不失真？

AI 20 请让 AI 帮助给出：在 Python 里调用 C 函数库的三种可行做法（ctypes、cffi、Cython 扩展各一段最小示例），并比较三者的调用开销与上手成本。

## 作业二：C 函数库与 Python 调用

使用 C 语言实现下列算法，制作**函数库**：

1. `issorted()` 函数检查有序性
2. 冒泡法
3. 插入排序
4. 3-路快速排序

使用 Python 从 C 编译的函数库中**调用** 4 种算法，测试它们对大（$n = 10^6$）、小（$n = 20$）随机数组的性能差异。

【还可以比较用 Python 实现的排序算法的速度，以及 Python 标准库的排序算法的速度。】

Q 18 AI 21



Q 18 n = 20 的数组上，四个算法的耗时都在微秒以下。这时测出来的差异主要来自算法本身，还是来自调用开销（函数调用、数据在两种语言之间搬运）？请给出你的判断依据。

AI 21 请让 AI 帮助估算：n = 10^6 时冒泡排序大约要做多少次比较、按每秒 10^9 次比较估算需要多长时间，并说明这个量级下作业里应该怎么安排测试，才能不把大部分时间花在等待上。

## 作业三：用指针数组搭一个上三角矩阵

这一次专门练 **C 的指针与动态内存**。要把一个上三角矩阵存成“每行一段连续数组、行与行用指针数组串起来”的结构。

**要求**

1. 只存非零部分：第 $i$ 行存第 $i$ 到第 $n$ 列的元素，行长度随行号递减（第 1 行 $n$ 个、第 2 行 $n-1$ 个……最后一行 1 个），总共 $n(n+1)/2$ 个元素。
2. 行与行之间用**指针数组**组织，例如

   ```
   typedef struct {
     int n;          /* 阶数 */
     double **rows;  /* rows[i] 指向第 i 行的数组，长度 n-i */
   } UpperTri;
   ```

   不允许用二维数组 `double a[n][n]`，也不允许用变长数组（VLA）。
3. 实现 `create(n)`、`destroy(a)`、`get(a,i,j)`、`set(a,i,j,v)`，下标从 0 开始，$i < j$ 的位置（下三角）一律视为 0、不占存储。
4. 实现三个乘法函数，并说明结果是什么结构：
   - 上三角 × **上三角**：结果仍是上三角，请只算需要的部分；
   - 上三角 × **下三角**：结果一般是普通方阵，请完整分配存储；
   - 上三角 × **普通矩阵**：结果一般是普通矩阵。
5. 做**复杂度对比**：把“利用三角结构只算需要的部分”与“先补成普通矩阵再做 $O(n^3)$ 相乘”两种做法，分别统计乘加次数，并实测 $n = 100, 500, 1000$ 的耗时，给出结论。



6. 做**正确性验证**：把你的三种乘法结果与普通矩阵乘法的结果逐元素对照（误差 $< 10^{-12}$），并验证 $(AB)^T = B^T A^T$ 这类恒等式。
7. **内存自证**：`destroy` 之后不能有泄漏。用 `valgrind --leak-check=full` 跑一遍，或者自己在 `malloc/free` 处计数并打印配平结果。

【选作】把同一个结构再用双向链表实现一遍（每行一个结点，结点里存行数组指针），比较两种实现在随机访问与遍历上的差别。

Q 19 AI 22

指向的第 i+1 行数组在内存里是相邻的吗？get(a,i,j) 需要几次指针跳跃？如果改成用一维数组加行偏移表，这两件事会怎么变？

## 参考资料

**AI 22** 请让 AI 帮助说明：valgrind 的 --leak-check=full 输出里 definitely lost、indirectly lost、still reachable 三类分别指什么，各自对应的代码错误长什么样，以及为什么用指针数组做二维结构最容易出现 indirectly lost。

## 参考资料

课程讲义尽可能做到自足，有问题请及时提问、反馈，找到错误会有奖励。

**参考资料：**

- 互动精致开源教材：靳宇栋，Hello, 算法！
- 国内新近高质量教材：俞勇 等，《数据结构》（101 计划）
- 国内优秀数据结构教材：邓俊辉，《数据结构（C++语言版・第 3 版）》[1]
- 国外优秀算法入门：A. Levitin, 《算法设计与分析基础》
- 国外优秀数据结构与算法：D. Reed, J. Zelle, 《数据结构和算法（Python 和 C++ 语言描述）》
- 经典算法教材：T. Cormen 等, 《算法导论（原书第 3 版）》[2]













Q 20 AI 23

[1]清华大学，邓老师的书与授课材料包括视频均由作者本人在其主页免费提供，强烈推荐！

[2]算法圣经，内容全面，适合有一定基础的同学深入学习。非常难！英文版已出版 4th edition, 中文版译自 3rd edition.



Q 20 上面六本书里，哪一本最适合在写本周作业时查阅“快速排序的分割实现”？请说明你选它的理由，并给出它可以查到的具体章节或关键词。

AI 23 请让 AI 帮助寻找并核对：两到三个可以交互观察排序过程的开源演示（网页动画或可视化库），说明它们各自支持哪些算法，并请 AI 说明为什么交互演示能帮助理解“输入分布影响性能”这件事。